SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: **ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS** **COURSE CODE:** **MLA01**

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

AI Model Design Exercise

Code of Conduct:

I _Juhi Surin (192525148)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Juhi Surin**

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE QUESTION

S.No.	Scenario / Problem Statement	AI Model / Technique to be Developed	Expected Development Outcome
1	**Smart Loan Approval:** A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history.	Decision Tree	Design and implement a decision-tree-based AI model and determine the most suitable attribute-selection measure such as Information Gain, Gain Ratio, or Gini Index.
2	**Student Performance Prediction:** A university wants to predict whether a student is likely to pass or fail a course using attendance, internal marks, assignment performance, study hours, and previous academic performance.	Inductive Learning	Construct a predictive model from training examples and derive generalized decision rules from the learned patterns.
3	**Disease Risk Prediction:** A healthcare system needs to classify patients into low-risk and high-risk categories based on clinical measurements.	Statistical Learning	Design a statistical learning model, train it using patient data, and evaluate its generalization ability on unseen data.
4	**Image Recognition System:** An organization wants to automatically classify images into categories such as vehicles, animals, and buildings.	Neural Network	Design and implement a neural-network classifier by specifying the network architecture, activation functions, loss function, optimizer, and training procedure.
5	**Handwritten Character Recognition:** A system must recognize handwritten characters from scanned images.	Multilayer Neural Network	Develop a multilayer neural network and demonstrate how backpropagation updates weights to minimize classification error.
6	**Non-Linear Classification:** A dataset contains two classes that cannot be separated using a straight-line decision boundary.	SVM with Kernel Method	Design an SVM-based classifier using an appropriate kernel function and justify the selected kernel based on the characteristics of the dataset.

7	**Intelligent Recommendation Agent:** An online platform wants an agent that learns which recommendations to provide based on user responses.	Reinforcement Learning	Design an RL model by defining the states, actions, rewards, policy, and learning strategy, and demonstrate how the agent learns from user feedback.
8	**Autonomous Game Agent:** An AI agent must learn to play a game where successful actions receive positive rewards and unsuccessful actions receive penalties.	Reinforcement Learning	Develop an RL-based game agent and demonstrate how its policy and performance improve through repeated interaction with the environment.
9	**Explainable Diagnosis System:** A medical AI system predicts a disease but must also provide an explanation for its prediction.	Explanation-Based Learning	Design an explanation-based learning approach using domain knowledge and training examples to generate an understandable justification for the prediction.
10	**AI Model Selection Challenge:** A company provides a dataset for predicting customer churn.	Comparative AI Model Design	Design and compare at least two learning approaches from decision trees, statistical learning, neural networks, or kernel methods. Select the best model based on accuracy, interpretability, computational requirements, and generalization ability.

Structure of the Report:

S.No.	Report Component	What the Student Should Include
1	**Problem Statement**	Clearly describe the real-world problem, input data, expected output, and AI task to be solved.
2	**Objectives**	State the specific goals of developing the AI model and expected outcomes.

3	**Dataset Description**	Provide dataset source, number of instances, features, target variable, data types, and important attributes.
4	**Data Preprocessing**	Explain data cleaning, missing-value handling, encoding, normalization/scaling, feature selection, and train-test splitting.
5	**Selected AI Technique and Justification**	Identify the selected AI technique and justify its suitability for the problem.
6	**Model Design / Architecture**	Present the model structure, components, input/output, parameters, hyperparameters, and architecture/flow diagram.
7	**Algorithm / Pseudocode**	Provide the algorithmic steps or pseudocode showing model training and prediction/decision-making.
8	**Implementation**	Provide programming language, libraries/tools, important code sections, and implementation details.
9	**Experimental Results**	Present results using tables, graphs, sample predictions, learning curves, confusion matrices, or other relevant outputs.
10	**Performance Evaluation**	Evaluate the model using appropriate performance metrics such as accuracy, precision, recall, F1-score, MSE, RMSE, or cumulative reward.
11	**Result Analysis**	Interpret results, identify important patterns, compare actual and predicted outcomes, and discuss strengths and weaknesses.
12	**Limitations**	Discuss limitations related to dataset, model complexity, computational resources, accuracy, generalization, or deployment.
13	**Future Enhancements**	Suggest improvements such as additional data, feature engineering, hyperparameter tuning, alternative algorithms, or deployment.
14	**Conclusion**	Summarize the problem, methodology, major findings, model performance, and overall effectiveness.
15	**References**	List datasets, research papers, books, websites, documentation, libraries, and other sources used.
16	**GitHub Repository Link**	Provide the public GitHub repository URL containing the complete source code, dataset information/source, implementation, results, README, and supporting files.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	**10%**	9–10% – Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.

2. Dataset Selection and Understanding	**10%**	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	**10%**	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	**10%**	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.
5. Model Design / Architecture	**10%**	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.
6. Algorithm / Pseudocode	**5%**	5% – Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4% – Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1% – Algorithm is missing or substantially incorrect.

7. Implementation and GitHub Repository	**15%**	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12% – Functional implementation and well-organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6% – Implementation is incomplete/non-functional or GitHub repository is missing.
8. Experimental Design and Results	**10%**	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8% – Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4% – Insufficient, incorrect, or missing experimental results.
9. Performance Evaluation	**10%**	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8% – Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4% – Inappropriate metrics or performance evaluation is missing.
10. Result Analysis and Interpretation	**5%**	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4% – Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1% – Results are not meaningfully analyzed.
11. Limitations and Future Enhancements	**5%**	5% – Clearly identifies realistic limitations and proposes technically relevant and feasible improvements.	4% – Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.

12. Documentation and Technical Presentation	**10%**	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				

SIMATS ENGINEERING

Assessment Tool 1: AI Model Design Exercise

Course: Artificial Intelligence and Expert Systems (MLA01)

Question Attempted: Q1 — Smart Loan Approval (Decision Tree)

Problem: A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history. This report designs and implements a decision-tree-based AI model and identifies the most suitable attribute-selection measure.

1. Problem Statement

Manual loan approval is slow, inconsistent, and prone to human bias. The bank requires an automated classification system that predicts loan approval (Approved / Rejected) from applicant attributes: income, credit score, employment status, loan amount requested, and repayment history. The AI task is a binary supervised-classification problem, and the output must also be interpretable, since loan decisions require justification to regulators and customers.

2. Objectives

- To design a Decision Tree classifier that predicts loan approval status from applicant data.
- To identify the most suitable attribute-selection measure (Information Gain, Gain Ratio, or Gini Index) for this dataset.
- To preprocess and encode categorical and numerical applicant data correctly.
- To train, test, and evaluate the model using standard classification metrics.
- To interpret the resulting tree and provide actionable, explainable decision rules for bank officers.

3. Dataset Description

A representative loan-application dataset of 800 records was used, with five input features and one target label.

Attribute	Type	Description
Income	Numeric	Applicant's monthly/annual income (₹10,000–₹150,000 range)
Credit Score	Numeric	Credit bureau score (300–850)
Employment Status	Categorical	Salaried / Self-Employed / Unemployed
Loan Amount	Numeric	Amount requested (₹20,000–₹600,000)
Repayment History	Categorical	Good / Average / Poor
Approved (Target)	Binary	1 = Approved, 0 = Rejected

Dataset size: 800 records (640 training / 160 testing, 80:20 split, stratified). Class balance: ~47% approved, ~53% rejected in both splits, so the classes are reasonably balanced and accuracy is a meaningful metric alongside precision/recall.

4. Data Preprocessing and Feature Engineering

- Missing-value check: no missing values were present; a median/mode imputation step is included in the pipeline for robustness to future data.
- Categorical encoding: Employment Status and Repayment History were label-encoded into numeric codes for tree splitting.
- Feature scaling: not required for Decision Trees, since splits are threshold-based and scale-invariant (unlike distance-based models).
- Train–test split: 80:20 stratified split to preserve class ratio in both sets.
- Outlier handling: income and loan amount were clipped to realistic bounds to prevent extreme values from dominating splits.

5. Selected AI Technique and Justification

A Decision Tree Classifier was selected because: (1) the problem needs interpretable, rule-based decisions that a bank can justify to customers/regulators; (2) it naturally handles a mix of numeric (income, credit score) and categorical (employment, repayment) attributes; (3) it requires no feature scaling; and (4) it directly supports comparison of attribute-selection measures, which the exercise requires.

Three attribute-selection measures were compared under identical settings (max depth = 5, random_state = 42):

Measure	Basis	Test Accuracy	F1-Score
Gini Index	Impurity / probability of misclassification	68.1%	0.675
Information Gain (Entropy)	Reduction in entropy after split	65.0%	0.606
Gain Ratio (approx. via log-loss criterion)	Entropy normalized by split information	65.0%	0.606

Result: the Gini Index gave the best test performance and was selected as the attribute-selection measure for the final model, since it produced higher accuracy and F1-score than entropy-based splitting on this dataset, while remaining computationally cheaper (no logarithm computation).

6. Model Design / Architecture

Architecture: a single Decision Tree Classifier, criterion = Gini, max_depth = 5 (to prevent overfitting), min_samples_split default. Input layer: 5 features (income, credit_score, employment_enc, loan_amount, repayment_enc). Each internal node splits on the feature/threshold that maximizes Gini-based impurity reduction. Leaf nodes output the majority class (Approved/Rejected) with an associated class probability.

Feature importance (Gini-based) obtained from the trained tree, showing which attributes drive decisions most:

Feature	Importance
Credit Score	0.313
Loan Amount	0.287
Repayment History	0.219
Income	0.136
Employment Status	0.046

Credit score and loan amount dominate the tree's early splits, which matches real-world lending logic — this validates the model's structural soundness.

7. Algorithm / Pseudocode

BUILD_DECISION_TREE(D, attributes):

- 1. If all records in D belong to one class, return a leaf node with that class.
- 2. If attributes is empty, return a leaf node with the majority class in D.
- 3. For each candidate attribute A, compute Gini(D, A) for every possible split threshold.
- 4. Select attribute A* and threshold t* that minimize weighted Gini impurity (maximize impurity reduction).
- 5. Create a decision node on A* at threshold t*; partition D into D_left (A* ≤ t*) and D_right (A* > t*).
- 6. Recursively call BUILD_DECISION_TREE on D_left and D_right until max_depth is reached or a stopping rule triggers.
- 7. Return the constructed tree.

PREDICT(tree, x): traverse from the root, at each node follow the branch matching x's attribute value/threshold, until a leaf is reached; output the leaf's class label.

8. Implementation

Language/Tools: Python 3, scikit-learn (DecisionTreeClassifier), pandas, NumPy for data handling, matplotlib for visualization. Key implementation steps:

- Load and clean data with pandas; encode categorical columns with LabelEncoder.
- Split data using train_test_split(test_size=0.2, stratify=y, random_state=42).
- Train DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42).
- Evaluate with accuracy_score, precision_score, recall_score, f1_score, confusion_matrix from sklearn.metrics.
- Visualize the tree with sklearn.tree.plot_tree and export decision rules with export_text.

The full implementation, dataset generation script, and results are organized in a public GitHub repository (see Section 16) containing model.py, README.md, and a results/ folder with plots and metric logs.

9. Experimental Design and Results

Experiment: an 80:20 stratified holdout split was used (640 training, 160 testing records). Three criteria (Gini, Entropy, Log-loss) were each trained and evaluated once on the identical split to isolate the effect of the attribute-selection measure. Confusion matrix for the selected Gini model on the test set:

	Predicted: Rejected	**Predicted: Approved**
Actual: Rejected	56	29
Actual: Approved	22	53

This shows 109 of 160 test cases (68.1%) correctly classified, with a reasonably even distribution of false positives (29) and false negatives (22) — i.e., the model does not systematically favor one class.

10. Performance Evaluation and Metrics

Metric	Value
Accuracy	68.1%
Precision (Approved)	0.646
Recall (Approved)	0.707
F1-Score	0.675

Recall (0.707) is higher than precision (0.646), meaning the model is slightly more inclined to approve borderline cases; for a bank, this trade-off could be tuned by adjusting the classification threshold depending on whether the business prioritizes minimizing bad loans (favor precision) or minimizing missed good customers (favor recall).

11. Result Analysis and Interpretation

The model performs well above the 50% random-guess baseline, confirming that income, credit score, employment, loan amount, and repayment history carry real predictive signal for loan approval. Credit score and loan amount are the strongest predictors, consistent with standard credit-risk practice. The moderate (not near-perfect) accuracy reflects realistic overlap between approved and rejected applicants near decision boundaries — exactly the borderline cases a human loan officer would also find difficult. Errors are balanced between false positives and false negatives, indicating no strong class bias in the trained tree.

12. Limitations

- Synthetic/representative data was used for demonstration; real bank data may contain additional attributes (existing debt, collateral, co-applicant income) that would improve accuracy.
- A single train-test split was evaluated; k-fold cross-validation would give a more robust estimate of generalization.
- Shallow tree depth (5) limits complexity and may underfit subtle interactions between features.
- The model does not currently correct for class imbalance beyond stratified splitting.

13. Future Enhancements

- Apply k-fold cross-validation and grid-search hyperparameter tuning (max_depth, min_samples_leaf) to improve robustness.
- Compare against ensemble methods (Random Forest, Gradient Boosting) to see if accuracy improves while retaining interpretability via feature importance.
- Incorporate additional real-world features (existing liabilities, collateral value, co-applicant details).
- Deploy the model behind a REST API with a monitoring dashboard for drift detection in production.

14. Conclusion

This report designed and implemented a Decision Tree–based AI model for automated Smart Loan Approval. Among the attribute-selection measures compared, the Gini Index produced the best performance (68.1% accuracy, F1 = 0.675) and was selected as the final splitting criterion. The resulting

tree is interpretable, highlights credit score and loan amount as the dominant decision factors, and provides a practical, explainable foundation that a bank could extend with real data, cross-validation, and ensemble methods for production use.

15. References

1. Scikit-learn Documentation — Decision Trees, https://scikit-learn.org/stable/modules/tree.html

2. Quinlan, J. R. "Induction of Decision Trees." Machine Learning, 1986.

3. Breiman, L. et al. "Classification and Regression Trees." Wadsworth, 1984.

4. Pandas Documentation, https://pandas.pydata.org/docs/

16. GitHub Repository Link

https://github.com/juhisakshisurin/MLA0101-AI-with-expert-systems